

AI Code Review & Rewrite Agent

Project Description:

CodeRefine is a Generative AI-powered web application designed to assist software developers throughout the code development lifecycle by combining intelligent code generation, automated code review, and interactive AI-driven assistance into a single, unified platform.

The application addresses two persistent challenges in modern software engineering: the time-intensive nature of manual code review and the inefficiency of writing repetitive boilerplate code from scratch. By leveraging Large Language Models through the Groq API (powered by LLaMA 3), CodeRefine enables developers to describe programming tasks in natural language and receive clean, well-documented, production-ready code across multiple languages including Python, JavaScript, TypeScript, Java, Go, Rust, and more.

Beyond code generation, the platform's core differentiator is its automated code optimization engine, which performs a structured analysis of submitted code across four configurable dimensions — bug detection, performance bottlenecks, security vulnerabilities, and adherence to best practices. The system returns categorized, severity-rated issues with line-specific explanations and actionable suggestions, alongside a fully rewritten, optimized version of the code and a quantitative quality score comparing the code's condition before and after refinement.

To support deeper engagement, CodeRefine includes a streaming AI assistant that allows developers to ask follow-up questions, request clarifications, and discuss optimization decisions in real time, using Server-Sent Events to deliver responses incrementally for a natural, conversational experience.

The application is built on a modern, scalable architecture: a FastAPI backend handles authentication, request routing, and AI orchestration, while a JavaScript and Tailwind CSS frontend delivers a clean, responsive, and visually polished user interface. Security is prioritized through JWT-based authentication and bcrypt password hashing, ensuring user data is handled safely.

A distinguishing design principle of CodeRefine is its domain-focused intelligence — a built-in topic-filtering mechanism ensures the AI assistant remains dedicated exclusively to programming and software development queries, declining unrelated requests to maintain reliability and purpose.

Overall, CodeRefine represents a practical demonstration of applying generative AI to real-world software engineering workflows, showcasing how AI can act as a collaborative tool that enhances developer productivity, improves code quality, and accelerates the development cycle — while keeping human judgment central to the final decision-making

process.

Scenario 1: Automated Code Review for Development Teams

Integrating the AI Code Review and Rewrite Agent into software development workflows can greatly improve productivity and code quality. The system's strong language understanding allows for automated analysis of codebases in several languages, including Python, Java, C++, and JavaScript. It accurately identifies bugs, performance issues, and security vulnerabilities, giving developers real-time insights and practical feedback. By automating the review process, development teams can rely less on manual peer reviews, keep coding standards consistent, and speed up release cycles. This automation lets developers concentrate on innovation and solving complex problems while ensuring their code stays optimized and ready for production.

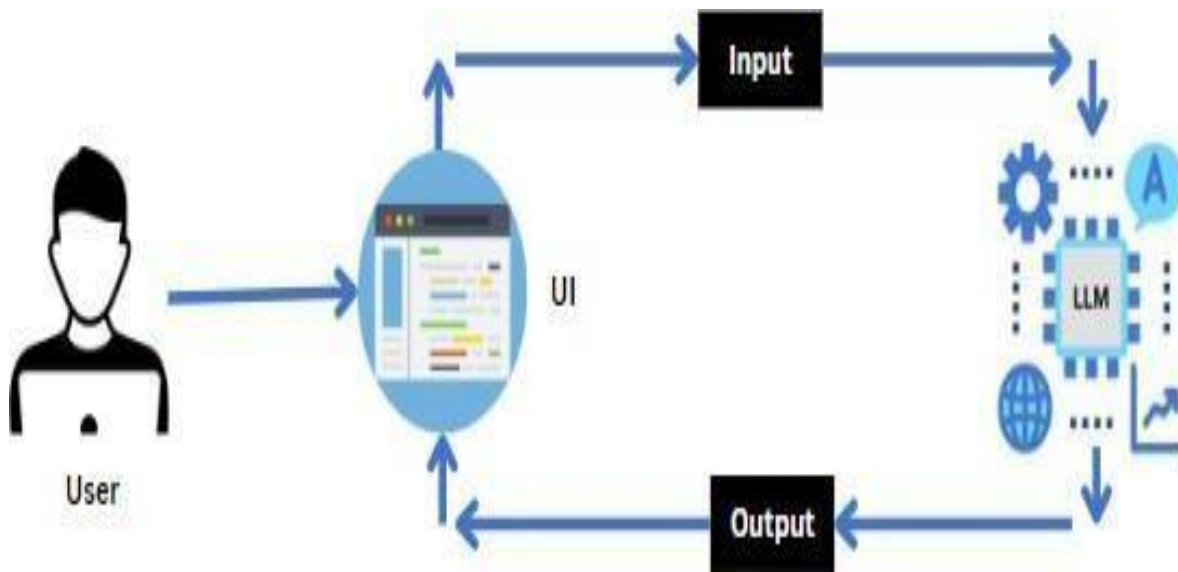
Scenario 2: Intelligent Learning Tool for Programming Education

The AI Code Review and Rewrite Agent can be integrated into educational platforms to help students and teachers learn programming best practices. By using its smart analysis and refactoring features, the platform can review student code submissions, point out errors, and offer detailed explanations along with improved versions of their code. This interactive method helps students grasp coding concepts, learn from their mistakes, and create clean, efficient programs. Teachers can use the tool to automate grading, give personalized feedback, and track student progress effectively. As a result, this AI-powered learning assistant supports hands-on learning, improves understanding, and builds coding skills in learners.

Scenario 3: Quality Assurance Enhancement for Software Enterprise

For software companies, the AI Code Review and Rewrite Agent provides a strong solution for keeping code quality high, secure, and efficient in large projects. The system constantly scans repositories, finds issues, and automatically improves code to meet industry standards. It makes sure that organizational policies are followed and boosts long-term maintainability. By using real-time analysis powered by Groq's fast inference, companies can improve their quality assurance processes, reduce bugs after deployment, and speed up time to market. This integration increases overall

development efficiency while ensuring the reliability and security of enterprise-grade software systems.



Architecture:

Project Flow

Users register/log in securely using JWT-based authentication, then input their source code and select a programming language from the dashboard.

The selected code and language are sent to the backend, which is powered by Groq's LLaMA 3.3 70B model, for real-time AI-driven analysis.

The system scans the code to detect bugs, security flaws, performance issues, and best-practice violations, classifying each issue by severity — Critical, High, Medium, Low.

The AI generates detailed, line-by-line improvement suggestions along with clear explanations, giving users instant feedback for quick understanding and resolution.

Users can trigger the Optimize & Rewrite function, which automatically refactors the code into a clean, well-documented, production-ready version, accompanied by a before/after quality score.

A dedicated AI Assistant chat allows users to ask follow-up questions, request clarifications, and discuss code improvements interactively, with responses streamed in real time using Server-Sent Events (SSE).

A built-in topic-filtering mechanism ensures the assistant responds only to programming-related queries, maintaining focus and reliability.

Built with FastAPI, HTML5, Tailwind CSS, and JavaScript, the platform delivers real-time code review with categorized issue cards, side-by-side optimized code output, one-click copy/export, and clear visual feedback through severity badges and quality score widgets.

To accomplish this, we completed the following activities:

- Model Initialization:

Generated a Groq API key

Initialized the pre-trained LLaMA 3.3 70B (Versatile) model for code generation and optimization

Configured the Mixtral 8x7B model for the streaming AI chat assistant

- Application Development:

Built secure backend APIs using FastAPI, including authentication (JWT + bcrypt), code generation, code optimization, and chat endpoints

Connected the backend to Groq for real-time inference with structured JSON responses

Designed an interactive, professional frontend using HTML5, Tailwind CSS, and JavaScript, featuring a 3D-animated login page, multi-tab dashboard, severity-based issue cards, and a live quality-score widget

Implemented a topic-guard filter to restrict the assistant to programming-related queries only

- Deployment:



Hosted the backend locally using Uvicorn with auto-reload for development, with provisions for cloud deployment (Render/Railway) for live access

Optimized API response handling and prompt design for fast, seamless, AI-driven code analysis and generation

Prior Knowledge:

You must have prior knowledge of the following topics to complete this project.

- **LLM & Groq API:**

A large language model (LLM) is an advanced AI system that uses deep neural networks with billions of parameters to analyze, understand, and generate human-like code or text. In this project, the Llama 3.3 70B model integrated with the Groq API enables ultra-fast inference for real-time code review and rewriting. Understanding how LLMs perform tasks such as code generation, refactoring, and semantic analysis is essential. Examples of such models include ChatGPT by OpenAI, Claude by Anthropic, and Gemini by Google.

<https://groq.com/>

<https://www.llama.com/>

- **FastAPI:**

Knowledge of building high-performance web backends using FastAPI, modern Python framework used for developing RESTful APIs. Understanding how to create endpoints, handle requests, and integrate AI models with APIs is important.

<https://fastapi.tiangolo.com>

- **Frontend Development (HTML5, Tailwind CSS, JavaScript):**

Familiarity with creating responsive, interactive web interfaces using HTML5, Tailwind CSS, and Vanilla JavaScript. Understanding how to structure layouts, style components, and manage user interactions is required.

<https://tailwindcss.com>

- **Code Rendering Libraries:**

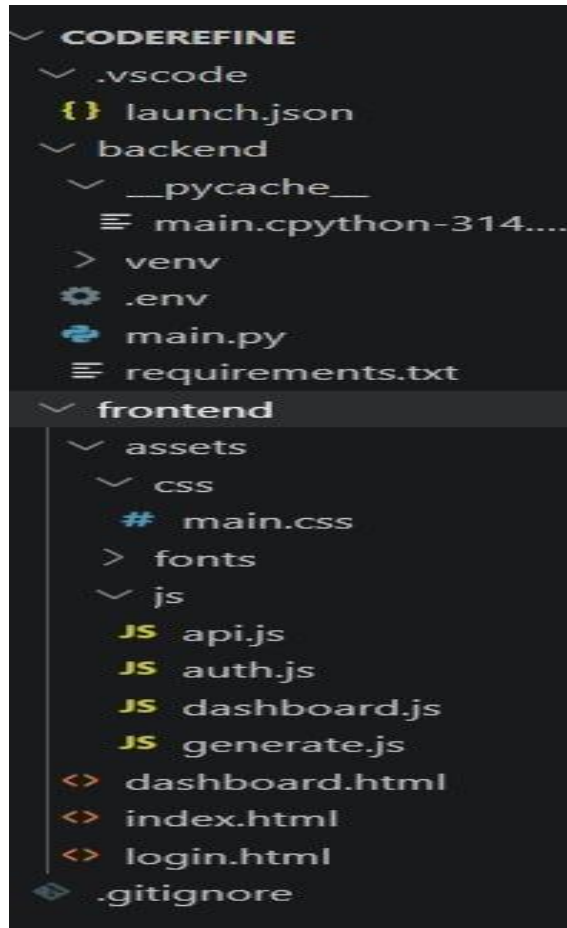
Basic knowledge of Marked.js and Highlight.js for rendering markdown content and syntax highlighting within the web interface. These libraries are used to display reviewed and rewritten code in a structured and visually clear manner.

<https://marked.js.org>

<https://highlightjs.org>

Project Structure:

Create the Project folder which contains application file as shown below



Milestone 1: Requirements Specification

Specifying the required libraries in the requirements.txt file ensures a smooth setup and reproducibility of the project environment. This makes it easier for others to copy the development environment.

Activity 1: Create a requirements.txt file to list the required libraries.

```
backend > ≡ requirements.txt
1  fastapi==0.115.0
2  uvicorn[standard]==0.30.1
3  python-dotenv==1.0.0
4  groq==0.13.0
5  httpx==0.27.2
6  python-multipart==0.0.9
7
```

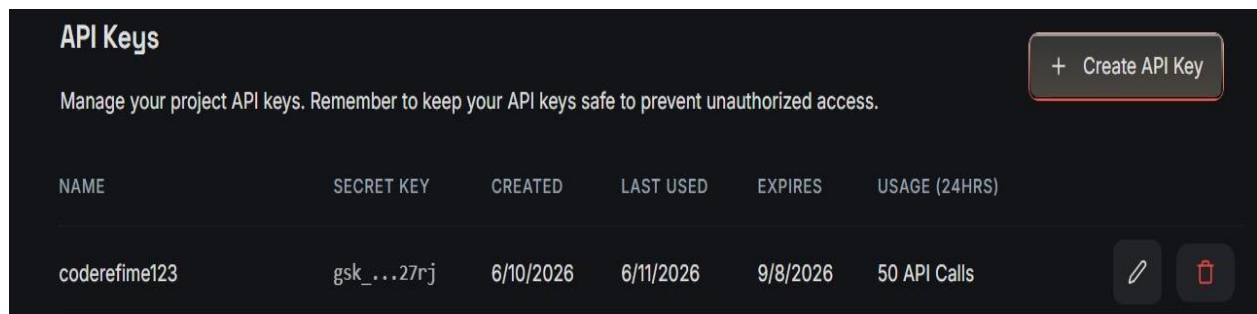
Activity 2: Install the required libraries.

```
(venv) PS C:\Users\dheer\Desktop\CodeRefine> pip install -r requirements.txt
```

Milestone 2: Initializing the Model

For initializing the model we need to generate GROQ API.

Activity 1: Generate GROQ API



Click on the link

(<https://in.search.yahoo.com/search?fr=mcafee&type=E210IN714G0&p=groq+api+key>)

- Then click on “API Keys-Groq Cloud”.
- Click on “Get API key” from the right navigation menu.
- Now click on “Create API key”. (Refer the above image)
- Copy the API key.

Activity 2: Initialize the pre-trained model

Activity 2.1: Import necessary files

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse
from pydantic import BaseModel
from groq import Groq
import os
from dotenv import load_dotenv
import re
```

- FastAPI is used to build high-performance backend APIs that manage code submission, analysis, and rewriting requests while enabling seamless communication between the frontend and the Groq-powered LLM for real-time processing and response delivery.
- Provides ultra-fast LLM inference using the **Llama 3.3 70B model** to analyze, review, and rewrite code instantly.

Activity 2.2: Configuration of the GROQ API:

Configuring the API key:

The configure function is used to set up or configure the GROQ API with an API key. The provided API key, in this case, is " " and it should be placed in ".env" file

```
GROQ_API_KEY="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"
```

Activity 2.3: Define the model to be used:

```
model="llama-3.3-70b-versatile",
temperature=0.3,
max_tokens=2000,
top_p=0.9
```

Generation Settings:

Defined model, temperature, max_tokens, and top_p parameters in the project configuration to control the output characteristics of the LLM responses.

Milestone 3: Interfacing with Pre-trained Model

In this milestone, we will build a prompt template to generate feedback based on the project details entered by the user.

Activity 1: Create a function to review response:

```
def parse_review_response(review_text: str) -> dict:
    """Parse the LLM response to extract structured data"""

    critical_section = re.search(r'### 🚨 Critical Issues.*?(?=###|\Z)', review_text, re.DOTALL)
    high_section = re.search(r'### 🟡 High Priority.*?(?=###|\Z)', review_text, re.DOTALL)
    medium_section = re.search(r'### 🟠 Medium Priority.*?(?=###|\Z)', review_text, re.DOTALL)
    low_section = re.search(r'### 🟢 Low Priority.*?(?=###|\Z)', review_text, re.DOTALL)

    critical_count = 0
    high_count = 0
    medium_count = 0
    low_count = 0
```

- The parse_review_response function is designed to extract and categorize feedback from an LLM-generated review text based on priority levels.
- The function accepts a review_text string as input and searches for sections labeled as Critical Issues, High Priority, Medium Priority, and

Low Priority using regular expression patterns.

- It initializes counters for each priority category and attempts to identify and separate these sections within the review text for structured analysis.
- The function is useful for converting unstructured LLM output into structured data, making it easier to quantify and further process review feedback.

Activity 2: Create a function to review code:

```
# — PROTECTED AI ROUTES —
@app.post("/analyze", response_class=AnalyzeRequestResponse)
async def analyze(req: AnalyzeRequest, username: str = Depends(verify_token)):
    prompt = build_analyze_prompt(req.code, req.language, req.checks)
    raw = await get_ai_response(prompt)
    data = parse_ai_json(raw)
    return AnalyzeResponse(
        score = int(data.get("score", 50)),
        summary = str(data.get("summary", "")),
        issues = data.get("issues", []),
        optimized_code = data.get("optimized_code", "")
    )

@app.post("/optimize")
async def optimize(req: OptimizeRequest, username: str = Depends(verify_token)):
    prompt = build_analyze_prompt(req.code, req.language,
        {"bugs": True, "performance": True, "style": True, "optimize": True})
    raw = await get_ai_response(prompt)
    data = parse_ai_json(raw)
    return data

@app.post("/security")
async def security(req: SecurityRequest, username: str = Depends(verify_token)):
    prompt = build_analyze_prompt(req.code, req.language,
        {"security": True, "bugs": True})
    raw = await get_ai_response(prompt)
    data = parse_ai_json(raw)
    return data

@app.get("/health")
def health():
    return {
        "status": "ok",
        "groq": bool(GROQ_API_KEY),
        "gemini": bool(GEMINI_API_KEY),
    }
```

- The `serve_tool` function serves the main tool page (`index.html`) after user login, returning the page content or an error message if the file is not found.
- The `review_code` function handles code review requests from the frontend and provides suggestions using the Groq API.
- It validates that the code input is not empty, raising an `HTTPException` if it is.
- The function formats the focus areas provided by the user, builds a prompt for the AI reviewer, and sends the request for review.
- The response is returned in the specified model format, making it suitable for structured API communication.

Milestone 4: Model Deployment

In this milestone, we are deploying the intelligent code review and rewrite model using FastAPI. The web interface visible in the image is built on FastAPI, providing a responsive and interactive platform where users can submit code, choose the programming language and focus areas, and receive structured review results directly in their browser. FastAPI enables efficient backend handling and smooth integration of the code review model into a web-based application, ensuring real-time analysis and recommendations for users.

Activity 1: Give the project title, display the image and the character and input information

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>🤖 AI Code Review & Rewrite Agent</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/highlight.js@11.9.0/styles/github.min.css">
  <script src="https://cdn.jsdelivr.net/npm/highlight.js@11.9.0/highlight.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/marked@4.3.0/marked.min.js"></script>
</head>
```

The web application sets the title to "AI Code Review & Rewrite Agent," which is clearly displayed at the top of the browser tab, helping users instantly understand the main purpose of the tool.

- A visually engaging robot emoji is placed next to the title, providing a friendly and welcoming impression for users interacting with the code review agent.
- The app uses multiple external style and script libraries, such as Tailwind CSS for modern UI design and Font Awesome for iconography, which enhance the application's look and user experience.
- Highlight.js and Marked.js are integrated to support syntax highlighting and formatted markdown rendering, ensuring that reviewed code and feedback are displayed in a readable and attractive format.
- Meta tags and responsive viewport settings in the HTML head ensure the web interface adapts seamlessly to different device sizes, offering a polished and user-friendly experience across all platforms.

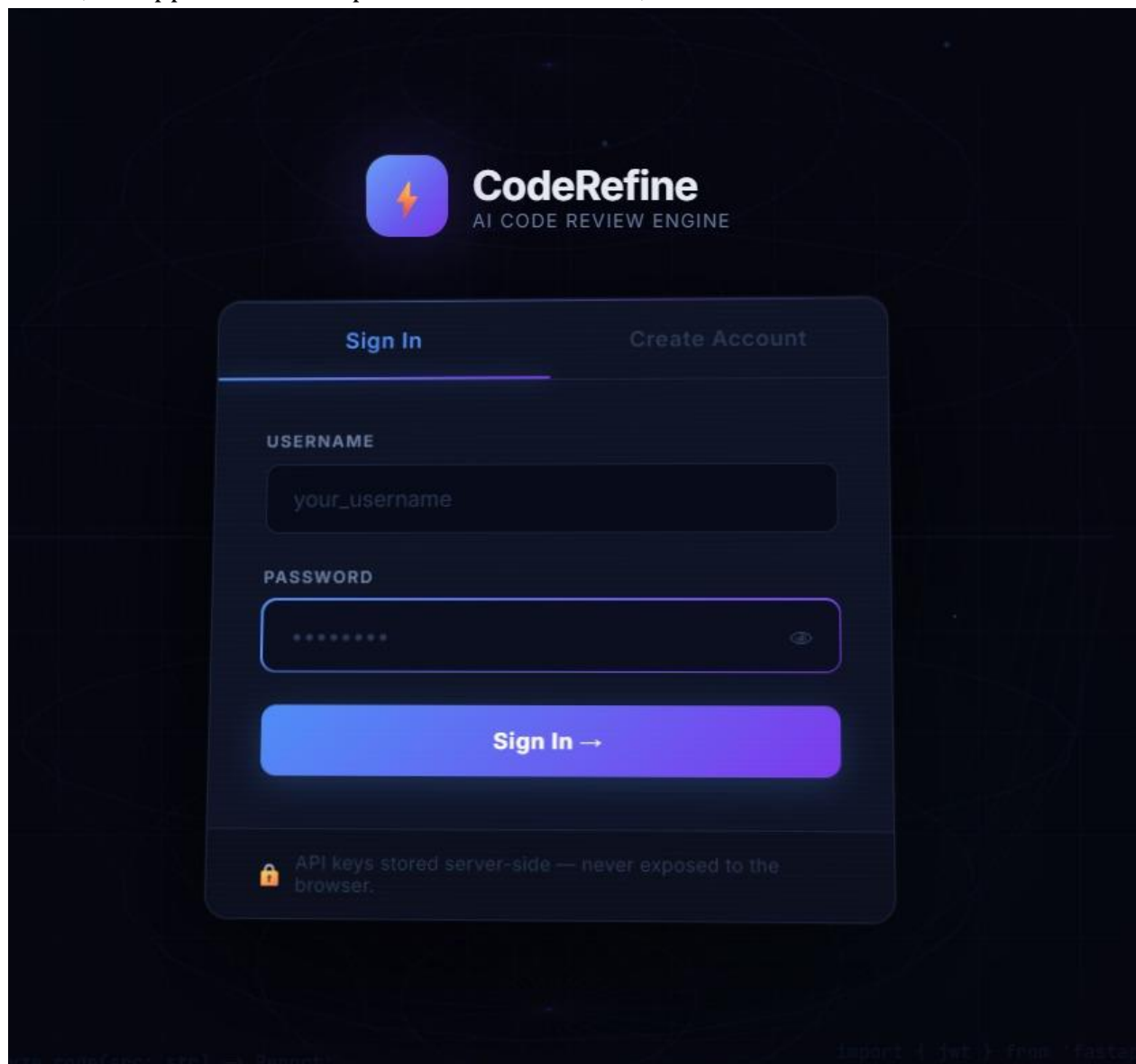
Activity 2: Run the web application

- Open the terminal from the vs code
- Navigate to the folder where your Python script is.
- Now type "python main.py" command
- Navigate to the localhost where you can view your web page

```
(venv) PS C:\Users\dheer\Desktop\CodeRefine> cd C:\Users\dheer\Desktop\CodeRefine\backend
>> venv\Scripts\activate
>> python -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\dheer\\Desktop\\CodeRefine\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [25700] using StatReload
INFO: Started server process [27636]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Activity 3: Output:

Now, the application will open in the web browser,



This is the output screen where functionalities are shown after giving input :

Code Review

Detect bugs, performance issues & bad practices

Load Sample

Clear

Analyze Code

Source Code

Python

Drop a file here or click to upload

code.py

35 lines · 933 chars

```
import sqlite3
import hashlib

def login(username, password):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    # BAD: SQL injection vulnerability
    query = "SELECT * FROM users WHERE username = '" + username + "'"
    cursor.execute(query)
    user = cursor.fetchone()
    conn.close()

    # BAD: MD5 is insecure for passwords
    hashed = hashlib.md5(password.encode()).hexdigest()
    if user and user[2] == hashed:
        return True
    return False

def get_all_users():
```

☒ Bugs ☒ Performance ☒ Security ☒ Style ☒ Rewrite

Analysis Results

Issues

Optimized

Diff

Copy

Export

20

Quality Score

The provided Python code has significant issues with security, performance, and style, requiring major refactoring to ensure reliability and efficiency.

2

HIGH

2

MEDIUM

1

LOW

5

TOTAL

SECURITY HIGH

Line 6

SQL Injection Vulnerability

The code is directly concatenating user input into SQL queries, making it vulnerable to SQL injection attacks.

Use parameterized queries or prepared statements to prevent SQL injection.

SECURITY HIGH

Line 11

Insecure Password Hashing

The code is using MD5 for password hashing, which is insecure and can be easily cracked.

Use a secure password hashing algorithm like bcrypt, scrypt, or Argon2.

PERFORMANCE MEDIUM

Line 17

Loading All Data into Memory

After giving the input of second functionality, the output is:

Code Optimizer
Get a clean, optimized rewrite of your code

Original Code

```
cursor = conn.cursor()
query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))
user = cursor.fetchone()
conn.close()

if user and bcrypt.checkpw(password.encode(), user[2].encode()):
    return True
return False

def get_all_users(limit=100, offset=0):
    users = []
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users LIMIT ? OFFSET ?", (limit,
offset))
    for row in cursor.fetchall():
        users.append(row)
    return users

def process_data(items):
    result = []
    for item in items:
        for sub in item:
            result.append(str(sub))
    return ",".join(result)
```

Optimized Output Copy

Optimized Code — Score: 60/100

```
import sqlite3
import bcrypt

class Database:
    def __init__(self, db_name):
        self.conn = sqlite3.connect(db_name)
        self.cursor = self.conn.cursor()

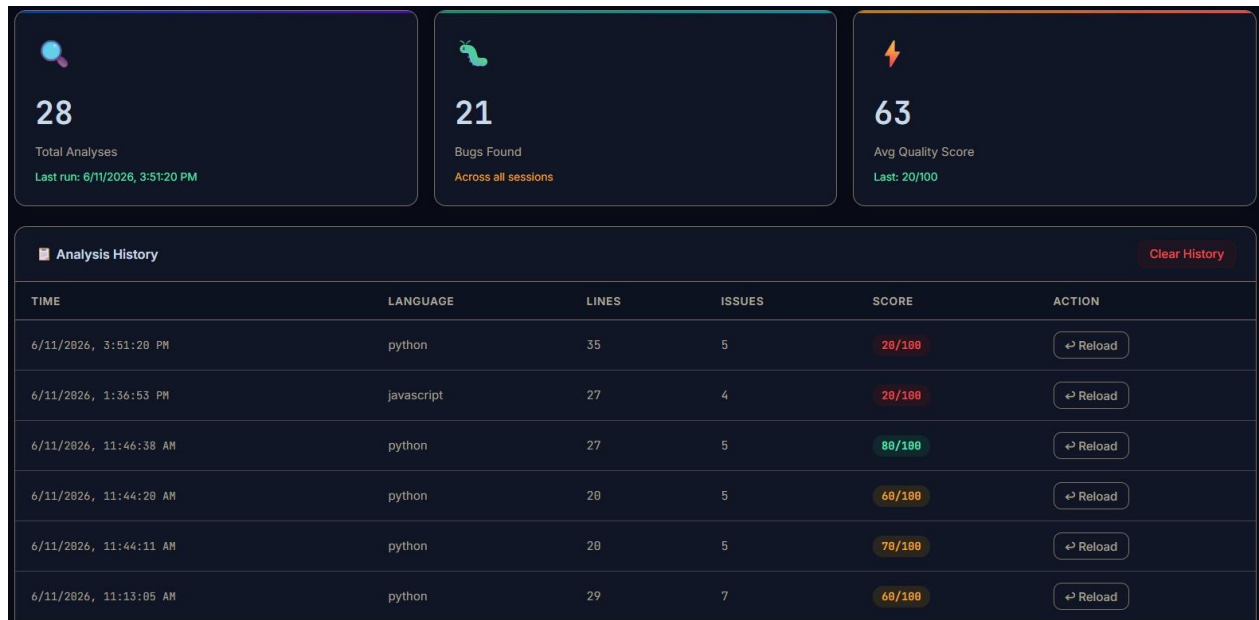
    def check_user_credentials(self, username, password):
        query = "SELECT * FROM users WHERE username = ?"
        self.cursor.execute(query, (username,))
        user = self.cursor.fetchone()
        if user and bcrypt.checkpw(password.encode(), user[2].encode()):
```

Fixed 5 issue(s) in the rewrite.

The Output generated with other functionality:



Dashboard:



History:

PYTHON 35 lines - 5 issues found The provided Python code has significant issues with security, performance, and style, requiring major refactoring to en...	6/11/2026, 3:51:20 PM	20/100
JAVASCRIPT 27 lines - 4 issues found The provided code is not written in JavaScript, but rather in Python, and has several issues with style, security, and b...	6/11/2026, 1:36:53 PM	20/100
PYTHON 27 lines - 5 issues found The provided Python code is a basic calculator with some error handling, but it can be improved in terms of performance,...	6/11/2026, 11:46:38 AM	80/100
PYTHON 20 lines - 5 issues found The provided Python code is a basic calculator implementation with some issues in terms of bugs, performance, security, ...	6/11/2026, 11:44:20 AM	60/100
PYTHON 20 lines - 5 issues found The provided Python code is a basic calculator implementation with some areas for improvement in terms of error handling,...	6/11/2026, 11:44:11 AM	70/100

Settings:

Settings

Configure backend connection and analysis preferences

AI Model Configuration



API keys are stored securely on the server

Your Groq, Gemini, and HuggingFace keys are configured in `backend/.env` and are **never** sent to or stored in the browser.

Backend URL

FastAPI server address

`http://localhost:8000`

Connection Status

Check if the backend is reachable



Test Connection

Analysis Preferences

Auto-detect Language

Detect language from code content



Include Optimized Rewrite

Generate a clean rewrite alongside review



Strict Security Mode

Flag even minor security concerns



Save History



Conclusion:

In conclusion, CodeRefine successfully demonstrates how Generative AI can be integrated into a practical, full-stack web application to assist software developers in their daily workflow. The project combines a FastAPI backend with a JavaScript and Tailwind CSS frontend to deliver a responsive and professional user experience. By integrating the Groq API running the LLaMA 3 model, the system provides three core capabilities: generating production-ready code from natural language prompts, analyzing existing code to detect bugs, performance issues, security vulnerabilities, and best-practice violations with an optimized rewrite and quality score, and an interactive AI assistant for real-time coding discussions through streaming responses.

The implementation of JWT-based authentication with bcrypt password hashing ensures secure user access, while the topic-filtering mechanism keeps the assistant focused strictly on programming-related queries. Through this project, I gained hands-on experience in API design, prompt engineering, asynchronous programming, and integrating large language models into real-world applications.

While the current version uses in-memory storage and prompt-based AI guidance rather than a fine-tuned model, it lays a strong foundation for future enhancements such as persistent databases, multi-agent architectures using frameworks like CrewAI, retrieval-augmented generation for documentation-aware suggestions, and cloud deployment for scalability. Overall, CodeRefine reflects how AI-powered tools can significantly reduce development time, improve code quality, and serve as a valuable learning aid for developers at all skill levels.